

Ce que le jury peut vous demander

Banque de questions et réponses préparées — PFE BTK

Toutes les réponses sont fondées sur ce que contient réellement le projet : code, base, datamart et rapport. Les questions marquées **(piège)** sont celles où une réponse trop assurée se retourne contre vous : la bonne attitude est d'annoncer la limite avant qu'on la trouve.

À savoir par cœur

Les six chiffres à ne jamais confondre : **45** agences dans le réseau, **49** entités dans l'extraction, **48** retenues dans la segmentation, **997** employés, **29 942** clients, **87,9 %** de taux de présence.

1 Contexte et cadrage

1.1 Pourquoi ce projet ? Quel problème résout-il ?

Les données du réseau étaient éclatées entre plusieurs fichiers et applications, et consolidées à la main. La direction ne disposait donc pas d'indicateurs fiables au moment de décider, et le suivi de présence n'avait pas de traçabilité. Le projet centralise la gestion du réseau et transforme ces données en indicateurs de pilotage.

1.2 Pourquoi un développement sur mesure plutôt qu'un progiciel ?

Un progiciel du marché est coûteux et peu adapté aux processus spécifiques de la BTK, notamment les circuits de validation à deux niveaux. Excel, lui, n'offre aucune traçabilité sur les opérations sensibles. Le sur-mesure permet de coller au métier réel et de garder le journal d'audit.

1.3 Quel a été votre périmètre exact, et ce que vous n'avez pas traité ?

Le périmètre couvre la gestion du référentiel, le pointage, les circuits de validation et la chaîne décisionnelle. N'ont pas été traités : le mobile, une API REST, l'authentification forte et l'industrialisation de l'ETL — ce sont les perspectives annoncées en conclusion.

1.4 Combien de temps a duré le stage ?

Quatre mois, du 16 février au 16 juin 2026, au département Data de la BTK.

2 Méthodologie

2.1 Pourquoi CRISP-DM et pas Scrum ?

Le projet est centré sur la donnée. Scrum organise la production logicielle mais ne dit rien de la compréhension ni de la valorisation des données. CRISP-DM est la méthodologie de référence des projets décisionnels, et ses six phases épousent le déroulement réel : compréhension métier, compréhension des données, préparation, modélisation, évaluation, déploiement.

2.2 CRISP-DM couvre-t-il le développement de l'application ?

(piège) Pas directement, et il faut le dire. CRISP-DM structure le volet données. Le développement applicatif a suivi des pratiques agiles — itérations et validations régulières avec le maître de stage — sans formaliser un cadre Scrum complet.

2.3 Différence entre méthode et méthodologie ?

La méthodologie est le cadre conceptuel qui guide le choix des pratiques ; la méthode est son application concrète. Scrum est une méthode issue de la méthodologie agile.

3 Conception et UML

3.1 Combien d'acteurs, et pourquoi ceux-là ?

Trois, qui correspondent exactement aux rôles de la table SECURITY_USERS : ADMIN, DIRECTEUR_COMMERCIAL et USER. Ce ne sont pas des acteurs théoriques : ce sont les valeurs réellement stockées et testées dans le code.

3.2 Pourquoi l'administrateur est-il relié aux treize cas d'utilisation ?

Parce qu'il couvre l'ensemble du périmètre fonctionnel : référentiel, sécurité, circuit des demandes, pointage et supervision du décisionnel. C'est ce que traduit le tableau des acteurs du rapport.

3.3 Pourquoi la relation « include » vers « S'authentifier » ?

Parce que l'accès à toute fonctionnalité exige une authentification préalable. La relation « include » factorise ce comportement une seule fois au lieu de le répéter sur les treize cas.

3.4 Différence entre include et extend ?

include est **obligatoire** : le cas inclus s'exécute systématiquement. extend est **conditionnel** : le comportement n'a lieu que dans certaines conditions. Ici l'authentification est toujours requise, donc include.

3.5 Pourquoi une association acteur-cas ne porte-t-elle pas de flèche ?

Parce qu'en UML, une association acteur-cas d'utilisation est un simple trait : elle exprime une participation, pas un sens de circulation. Seules les dépendances « include » portent une flèche ouverte.

3.6 Le directeur commercial peut-il gérer les agences ?

Non. Il **soumet** des demandes de création ; c'est l'administrateur qui valide et applique. Le contrôle est fait côté serveur : la page de validation des demandes de création n'accepte que le rôle ADMIN.

4 Architecture applicative

4.1 Décrivez votre architecture.

Quatre couches : Présentation (JSP), Contrôle (servlets), Métier (EJB), Persistance (JPA/Hibernate). Physiquement, trois niveaux : poste client, serveur WildFly, serveur Oracle. Power BI et la chaîne ETL se branchent directement sur les vues décisionnelles de la base.

4.2 Qu'est-ce qui garantit que les couches ne se mélangent pas ?

Trois faits vérifiables : aucun servlet n'accède à la base — ils n'ont que des services injectés ; aucun service ne manipule la requête HTTP ; aucune page ne contient de code Java. Chaque couche ne connaît que la suivante.

4.3 Est-ce du MVC ?

C'est une architecture en couches inspirée du MVC : Présentation = View, Contrôle = Controller, et le Model est affiné en Métier + Persistance.

4.4 Que se passe-t-il quand je clique sur « Agences » ?

Le filtre s'exécute d'abord et prépare les compteurs du bandeau. Le serveur route vers le contrôleur associé à cette adresse. Celui-ci vérifie la session et le rôle, demande la liste au service métier, qui interroge la base par Hibernate. Le résultat est passé à la page, qui produit le HTML.

4.5 Pourquoi les pages sont-elles dans un dossier protégé ?

Parce que le serveur en interdit l'accès par une adresse directe : on ne peut y entrer que depuis un contrôleur, donc après le contrôle de session et de rôle. Seules la page de connexion et celle de mot de passe oublié sont en accès libre, et c'est voulu.

5 Base de données et persistance

5.1 Combien de tables, et lesquelles ?

Seize entités correspondent à seize tables : le référentiel (AGENCE, B_UTILISATEURS, B_CLIENTS, CLIENT_BTK, B_BANQUE, GESTIONNAIRE), l'activité (B_OBJECTIF, POINTAGE), les circuits de demande (quatre tables), et le socle (SECURITY_USERS, NOTIFICATION, JOURNAL_ACTION).

5.2 Qui gère les transactions ?

Le serveur. L'unité de persistance est en JTA et les services sont des EJB : une transaction est ouverte à l'entrée de chaque méthode métier et validée à la sortie, ou annulée si une erreur survient. Aucune transaction n'est écrite à la main dans le code.

5.3 Comment évitez-vous l'injection SQL ?

Aucune requête n'est construite en collant des morceaux de texte. Les valeurs passent par des paramètres nommés, donc par des variables liées : elles ne sont jamais interprétées comme du code.

5.4 Hibernate crée-t-il vos tables ?

Non, et c'est volontaire. La création du schéma est désactivée : les tables sont créées par les scripts SQL du projet, et la base fait autorité. Une génération automatique pourrait modifier une structure existante sans contrôle.

5.5 Pourquoi une table de sécurité séparée des employés ?

Parce qu'un compte de connexion et un employé ne sont pas la même chose : tous les employés n'ont pas forcément de compte, et un compte porte un rôle applicatif. La connexion rattache les deux.

6 Sécurité

6.1 Comment gérez-vous les droits ?

Le rôle est déterminé à la connexion, depuis la table de sécurité, et conservé en session. Chaque page sensible le vérifie côté serveur avant d'afficher quoi que ce soit. Le menu masque en plus les entrées interdites, mais ce n'est qu'un confort d'affichage.

6.2 Et si quelqu'un contourne le menu et tape l'adresse directement ?

Il est renvoyé à la page de connexion : le contrôle est dans le contrôleur, pas dans le menu. Sur le circuit de validation, une troisième barrière existe dans le service, qui vérifie l'état de la demande avant d'agir.

6.3 Vos mots de passe sont-ils protégés ?

(piège) Non, et il vaut mieux le dire avant qu'on le trouve : le mot de passe est comparé en clair. C'est un choix de périmètre, pas un oubli. La correction est localisée dans une seule méthode : stocker un condensat salé, charger l'utilisateur par son seul identifiant, puis comparer le condensat.

6.4 Y a-t-il une traçabilité des actions ?

Oui : chaque opération sensible est écrite dans le journal d'audit avec l'auteur, son rôle, l'action, l'objet concerné et un libellé. C'est ce qui manquait aux fichiers Excel de départ.

6.5 Toutes les pages sont-elles protégées de la même façon ?

(piège) Non. Quatre pages de consultation ne vérifient que l'authentification, sans le rôle : l'annuaire des employés, les objectifs et deux écrans hérités. L'application est donc plus permissive que la matrice des droits du rapport sur ces écrans. Le correctif est le même contrôle de rôle que sur les autres pages.

7 Le front

7.1 Quelles technologies pour l'interface ?

Des pages JSP avec JSTL, une feuille de style écrite à la main, et deux bibliothèques : ApexCharts pour les graphiques et Lucide pour les icônes. Le HTML est produit par le serveur.

7.2 Pourquoi pas React ou Angular ?

Les écrans sont des listes et des formulaires : un client riche n'apporterait rien et ajouterait une chaîne de construction, un gestionnaire de paquets et des dépendances à maintenir. Le rendu côté serveur suffit et reste cohérent avec la pile Jakarta EE.

7.3 Comment les graphiques reçoivent-ils les données ?

Le contrôleur calcule les valeurs et les transmet à la page, qui les insère dans la configuration du graphique. Il n'y a pas d'appel asynchrone : la page arrive complète.

7.4 Votre interface est-elle responsive ?

La mise en page utilise des grilles souples et des unités relatives ; elle s'adapte aux tailles d'écran de bureau. Elle n'a pas été conçue pour le mobile, qui figure dans les perspectives.

8 Les fonctionnalités

8.1 Comment fonctionne le pointage ?

L'employé enregistre son arrivée d'un clic, puis son départ. Le statut est calculé par le serveur : au-delà de neuf heures, c'est un retard. L'administrateur peut saisir ou corriger un pointage, et une colonne distingue le pointage automatique de la saisie manuelle, ce qui conserve la traçabilité de la correction.

8.2 Que se passe-t-il si on clique deux fois sur « arrivée » ?

Rien n'est écrasé : l'heure déjà enregistrée est conservée. De même, un départ sans arrivée préalable ne crée aucun enregistrement.

8.3 Expliquez le circuit de validation à deux niveaux.

Une demande de modification part en attente. Le directeur commercial l'approuve — elle passe alors en cours — ou la refuse. Si elle est approuvée, l'administrateur l'exécute, et c'est seulement à ce moment que la donnée est réellement modifiée. Trois barrières : l'affichage du bouton, le croisement action-rôle dans le contrôleur, et la vérification de l'état dans le service.

8.4 Pourquoi deux validations et pas une seule ?

Pour séparer la décision de l'exécution. Le directeur juge de l'opportunité, l'administrateur applique techniquement. Aucune donnée sensible ne dépend d'une seule personne.

8.5 Que se passe-t-il à l'approbation d'une demande de création ?

L'objet est réellement créé : à l'approbation d'une demande d'employé, le compte employé est enregistré ; pour un client, la fiche client est créée avec le statut actif. Le demandeur est notifié et l'action est journalisée.

9 La chaîne ETL

9.1 Qu'est-ce qu'un ETL, et pourquoi en avoir fait un ?

Extraire, transformer, charger. Il consolide les données opérationnelles en un jeu agrégé par agence, contrôlé et reproductible, qui sert de socle à la segmentation. Power BI restitue le détail ; l'ETL prépare un jeu stable pour l'analyse.

9.2 Détaillez les cinq étapes.

Collecte des cinq tables sources ; nettoyage, qui écarte les lignes sans agence de rattachement ; transformation, qui agrège par agence ; intégration, qui fusionne les résultats sur la clé de l'agence ; contrôle de qualité, qui vérifie l'absence de valeur manquante, de valeur négative et de doublon — et qui interrompt le chargement si l'un de ces contrôles échoue.

9.3 Que produit exactement la chaîne ?

Un datamart en étoile : une table de faits au grain de l'agence avec sa dimension, et une seconde table de faits au grain agence $\times$ gestionnaire avec la sienne. Plus le fichier consommé par la segmentation.

9.4 Combien d'agences en sortie ?

49 entités : les 45 agences du réseau, plus le siège, deux structures régionales et deux agences premium.

9.5 Votre ETL est-il automatisé ?

(piège) Il est **rejouable** — une commande le relance de bout en bout et produit le même résultat — mais il n'est pas **planifié** : aucune exécution automatique n'est programmée. C'est précisément la perspective « industrialisation de l'ETL » annoncée en conclusion.

10 Modélisation décisionnelle et Power BI

10.1 Pourquoi une étoile plutôt qu'un flocon ou une constellation ?

Le besoin porte sur des grains homogènes et un nombre réduit d'axes d'analyse. Le flocon n'apporterait aucun gain d'espace significatif tout en alourdissant les requêtes ; la constellation, pensée pour plusieurs processus de faits, serait surdimensionnée. L'étoile offre la simplicité et la lisibilité attendues d'un outil en libre-service.

10.2 Combien de dimensions dans votre datamart ?

Deux : la dimension agence et la dimension gestionnaire. Les axes d'analyse cités pour justifier le choix de l'étoile — agence, période, gestionnaire, type de client — ne sont pas quatre tables : ce sont les angles sous lesquels on interroge les faits.

10.3 À quoi servent les vues V_BI_* ?

Ce sont la couche sémantique : elles traduisent les colonnes techniques en indicateurs métier, remplacent les clés par des libellés, ramènent les valeurs nulles à zéro et font les jointures une fois pour toutes. Power BI consomme du sens, pas de la structure — et si le schéma évolue, seule la vue change.

10.4 Pourquoi un schéma séparé pour Power BI ?

Pour que le compte de restitution ne puisse pas écrire dans les tables de production, et pour que les requêtes du rapport ne pèsent pas sur l'application.

10.5 Qu'est-ce qu'une mesure DAX ?

Un calcul défini dans Power BI, évalué à l'affichage dans le contexte des filtres actifs, qui produit dynamiquement un indicateur.

10.6 Mesure ou colonne calculée ?

La colonne calculée est évaluée à l'actualisation et stockée : elle sert à filtrer ou grouper. La mesure est évaluée à l'affichage : elle sert à agréger. Dans mon modèle, le rôle est une colonne calculée, le taux de présence une mesure.

10.7 Donnez trois mesures de votre rapport.

Le nombre distinct d'employés, le total des comptes ouverts — somme des trois types de comptes — et le taux de présence, rapport entre les pointages « présent » ou « retard » et le total des pointages.

10.8 Combien de pages, et que montrent-elles ?

Quatre : un résumé du réseau, l'analyse du portefeuille clients, la performance commerciale, et le suivi de présence. Des segments communs — district, période, type de client — filtrent l'ensemble.

10.9 Vos données Power BI sont-elles en temps réel ?

(piège) Non : elles sont importées, donc rafraîchies à la demande. Le temps réel est assuré par le tableau de bord embarqué dans l'application, qui interroge la base à chaque affichage. Les deux restitutions sont complémentaires.

11 Segmentation

11.1 Qu'est-ce que l'apprentissage non supervisé ?

Un apprentissage sans étiquette connue : on ne dit pas à l'algorithme à quel groupe appartient chaque agence, il découvre lui-même les regroupements à partir des ressemblances entre indicateurs.

11.2 Comment fonctionne K-Means ?

On fixe le nombre de groupes. L'algorithme place des centres, affecte chaque agence au centre le plus proche, recalcule les centres comme moyennes de leurs membres, et recommence jusqu'à stabilisation.

11.3 Pourquoi écarter le siège ?

540 employés contre une dizaine pour une agence : c'est une structure centrale, pas une agence. Le garder créerait un groupe d'un seul point et écraserait les distances entre toutes les autres. Il reste dans le datamart, il est seulement retiré de l'analyse.

11.4 Pourquoi passer les montants au logarithme ?

Parce que leur distribution est très asymétrique : quelques structures affichent des montants sans commune mesure avec les autres. Le logarithme ramène ces écarts à une échelle comparable, sans supprimer l'information.

11.5 Quels sont vos résultats ?

Trois groupes : 34 agences d'exploitation, 6 entités de production sans portefeuille client, et 8 entités sans activité enregistrée.

11.6 Vos groupes sont-ils « grandes, moyennes, petites » agences ?

(piège) Non, et c'est plus intéressant que cela. La segmentation ne sépare pas par taille mais par **nature d'activité** : celles qui exploitent un portefeuille, celles qui produisent sans portefeuille, et celles sans activité enregistrée. C'est un résultat que la direction ne lisait pas dans ses tableaux.

11.7 Comment savez-vous que votre segmentation est bonne ?

Par trois indices : la silhouette (0,604 pour K-Means), Davies-Bouldin et Calinski-Harabasz. Et par l'interprétabilité : les trois groupes correspondent à des réalités que le métier reconnaît.

11.8 Que feriez-vous de ces résultats ?

Différencier les objectifs commerciaux selon le profil, concentrer l'accompagnement sur les structures sans activité enregistrée, et vérifier si certaines d'entre elles relèvent d'un problème de saisie plutôt que d'activité.

12 Tests, déploiement et limites

12.1 Comment avez-vous testé ?

Par dix scénarios fonctionnels couvrant l'authentification, les rôles, le référentiel, le pointage, les demandes, l'ETL et le décisionnel. Tous conformes. Les plus parlants : le pointage après neuf heures qui bascule en retard, et l'accès refusé à un utilisateur simple sur une page réservée.

12.2 Avez-vous des tests automatisés ?

(piège) Non. Les tests sont manuels et documentés. Les services étant sans état, ils se prêteraient bien à des tests unitaires avec une base en mémoire : c'est la première chose que j'ajouterais.

12.3 Comment déploie-t-on l'application ?

Une commande produit l'archive, qu'on dépose sur le serveur WildFly. Les identifiants de la base ne sont pas dans l'archive : ils appartiennent à la ressource déclarée côté serveur, ce qui permet de déployer la même archive sur plusieurs environnements sans recompiler.

12.4 Que faites-vous si l'application ne se connecte plus à la base ?

Je distingue d'abord la cause : un test de connexion en dehors du serveur me dit si le problème vient d'Oracle ou de la configuration de la ressource. Si Oracle répond, c'est la ressource : compte, mot de passe ou nom de service à corriger, puis rechargement de la configuration.

12.5 Quelles sont les limites de votre travail ?

Le mot de passe comparé en clair, quatre écrans de consultation qui ne vérifient que l'authentification, l'absence de tests automatisés, et un ETL rejouable mais non planifié. Chacune a une correction identifiée et localisée.

12.6 Si vous deviez recommencer, que changeriez-vous ?

Je commencerais par le condensat des mots de passe et un contrôle de rôle centralisé dans un filtre plutôt que répété dans chaque contrôleur — cela supprimerait d'un coup les écarts entre écrans. Et j'écrirais les tests unitaires au fur et à mesure plutôt qu'à la fin.

13 Les questions qui déstabilisent, et quoi faire

Situation	Réaction
Vous ne connaissez pas la réponse	Le dire simplement, puis proposer ce que vous savez d'approchant. « <i>Je ne l'ai pas mis en œuvre, mais le principe est ...</i> » Ne jamais inventer un chiffre.
On conteste un de vos chiffres	Demander de quelle page il s'agit, et repartir de la source : « <i>ce chiffre vient du datamart, page tant</i> ». Vos chiffres sont vérifiables, c'est votre force.
On pointe une limite que vous connaissez	Confirmer aussitôt, donner la cause et la correction. Une limite reconnue et chiffrée est un signe de maîtrise.
On vous demande un détail technique très fin	Répondre au niveau que vous maîtrisez, sans bluffer. « <i>Ce que je peux vous dire précisément, c'est ...</i> »
La démonstration échoue	Ne pas s'excuser longuement. Annoncer la cause si vous la connaissez, montrer les captures du rapport, et poursuivre.
On vous coupe	Laisser finir la question, reformuler en une phrase pour vérifier que vous l'avez comprise, puis répondre.

À savoir par cœur

Trois réflexes : **ne jamais inventer un chiffre** — dire « je vérifie dans le rapport » vaut mieux qu'une approximation ; **annoncer vos limites avant qu'on les trouve** ; et **revenir à la donnée** — tout ce que vous avancez est vérifiable dans le rapport, le datamart ou le code.